Prathamesh Brahma CS23179 Assignment 7

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomforestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path="/content/drive/MyDrive/ML_Data/Student_Performance_dataset_5000 - Student_Performance_dataset_5000.csv"
df=pd.read_csv(path)
df.head(5)
```

	student_id	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours
0	1	18	Other	High School	7.84	1.56	2.20	3.05	2.11
1	2	18	Other	High School	2.21	2.22	2.10	1.65	2.55
2	3	22	Male	High School	3.45	0.00	0.29	1.34	2.06
3	4	17	Other	High School	5.75	2.08	3.01	2.27	2.20
4	5	19	Other	High School	0.83	1.72	3.33	2.85	0.70

5 rows × 21 columns

```
df.shape
```

```
(5000, 21)
```

```
df.describe()
```

	student_id	age	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep_hou
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.0000
mean	2500.500000	20.520400	4.539594	2.478734	2.011984	2.998086	1.564514	7.0164
std	1443.520003	2.870406	1.821665	1.177990	0.583906	1.467949	1.110807	1.1636
min	1.000000	16.000000	0.000000	0.000000	0.000000	0.000000	0.000000	4.0000
25%	1250.750000	18.000000	3.250000	1.660000	1.320000	1.990000	0.670000	6.2375
50%	2500.500000	20.000000	4.530000	2.480000	2.010000	2.980000	1.490000	7.0100
75%	3750.250000	23.000000	5.760000	3.290000	2.690000	4.030000	2.340000	7.8100
max	5000.000000	25.000000	11.840000	7.410000	8.000000	8.280000	5.640000	10.0000

```
df.columns
```

```
Index(['student_id', 'age', 'gender', 'academic_level', 'study_hours',
       'self_study_hours', 'online_classes_hours', 'social_media_hours',
       'gaming_hours', 'sleep_hours', 'screen_time_hours', 'exercise_minutes',
       'caffeine_intake_mg', 'part_time_job', 'upcoming_deadline',
       'internet_quality', 'mental_health_score', 'focus_index',
       'burnout_level', 'productivity_score', 'exam_score'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.isnull().sum()
```

Show hidden output

```
df[df.isnull().any(axis=1)]
```

student_id	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours

0 rows × 21 columns

```
X = df.drop(columns=['exam_score'])
y = df['exam_score']
```

```
X.head()
```

	student_id	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours
0	1	18	Other	High School	7.64	1.56	2.20	3.05	2.19
1	2	18	Other	High School	2.21	2.22	2.10	1.65	2.55
2	3	22	Male	High School	3.45	0.00	0.29	1.34	2.08
3	4	17	Other	High School	5.75	2.08	3.01	2.27	2.20
4	5	19	Other	High School	6.83	1.72	3.33	2.65	0.70

```
X = pd.get_dummies(X, dtype=int)
```

```
X.shape
```

```
(5000, 26)
```

```
X.head(5)
```

	student_id	age	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep_hours	screen_ti
0	1	18	7.64	1.56	2.20	3.05	2.19	6.52	
1	2	18	2.21	2.22	2.10	1.65	2.55	5.97	
2	3	22	3.45	0.00	0.29	1.34	2.08	8.39	
3	4	17	5.75	2.08	3.01	2.27	2.20	6.31	
4	5	19	6.83	1.72	3.33	2.65	0.70	8.01	

5 rows × 26 columns

```
y.head()
```

	exam_score
0	50.16
1	1.00
2	18.30
3	9.37
4	27.81

dtype: float64

```
X.columns
```

```
Index(['student_id', 'age', 'study_hours', 'self_study_hours',
       'online_classes_hours', 'social_media_hours', 'gaming_hours',
       'sleep_hours', 'screen_time_hours', 'exercise_minutes',
       'caffeine_intake_mg', 'part_time_job', 'upcoming_deadline',
```

```
       'mental_health_score', 'focus_index', 'burnout_level',
       'productivity_score', 'gender_Female', 'gender_Male', 'gender_Other',
       'academic_level_High School', 'academic_level_Postgraduate',
       'academic_level_Undergraduate', 'internet_quality_Average',
       'internet_quality_Good', 'internet_quality_Poor'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train,y_test = train_test_split(X,y,train_size=0.7,random_state=42)
```

```
X_train.shape,X_test.shape
```

```
((3500, 26), (1500, 26))
```

```
X_train.head()
```

	student_id	age	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep_hours	screen
1840	1841	17	3.96	2.87	0.31	2.62	2.63	5.86	
2115	2116	16	6.94	1.04	3.60	5.46	1.12	8.26	
4437	4438	24	6.09	3.55	4.73	2.87	1.42	4.69	
1146	1147	17	7.78	2.22	3.17	4.24	2.17	6.67	
2486	2487	17	7.95	3.80	1.39	3.00	1.82	6.92	

5 rows × 26 columns

```
from sklearn.preprocessing import StandardScaler
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

```
array([[-0.47189175, -1.23291707, -0.32898634, ..., -0.70045009,
         1.37310163, -0.69275929],
       [-0.28001648, -1.58340519,  1.3038051 , ..., -0.70045009,
         1.37310163, -0.69275929],
       [ 1.34010849,  1.22049978,  0.838076  , ..., -0.70045009,
        -0.72827821,  1.44350284],
       ...,
       [ 0.40166401, -1.23291707,  1.09011763, ..., -0.70045009,
        -0.72827821,  1.44350284],
       [ 0.87611921,  0.16903541, -0.21940302, ..., -0.70045009,
        -0.72827821,  1.44350284],
       [-1.15566543, -0.88242895, -0.32898634, ..., -0.70045009,
        -0.72827821,  1.44350284]])
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

model = RandomForestRegressor(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print('Mean Squared Error:')
print(mean_squared_error(y_test, y_pred))
```

```
Mean Squared Error:
25.823733582779997
```

```
bins = [0, 10, 20, 40, 65]
labels = ['Fail', 'Pass', 'Good', 'Excellent']
y_classified = pd.cut(y, bins=bins, labels=labels, right=False)
print(y_classified.value_counts())
```

```
exam_score
Good         1980
```

```
Pass         1454
Fail         1325
Excellent     241
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X, y_classified, test_size=0.3, random_stat

X_train_cls = scaler.transform(X_train_cls)
X_test_cls = scaler.transform(X_test_cls)

print("Shape of X_train_cls:", X_train_cls.shape)
print("Shape of X_test_cls:", X_test_cls.shape)
print("Shape of y_train_cls:", y_train_cls.shape)
print("Shape of y_test_cls:", y_test_cls.shape)
```

```
Shape of X_train_cls: (3500, 26)
Shape of X_test_cls: (1500, 26)
Shape of y_train_cls: (3500,)
Shape of y_test_cls: (1500,)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Initialize and train the RandomForestClassifier
model_cls = RandomForestClassifier(random_state=42)
model_cls.fit(X_train_cls, y_train_cls)

# Make predictions on the test set
y_pred_cls = model_cls.predict(X_test_cls)

# Evaluate the model
print("Accuracy Score:", accuracy_score(y_test_cls, y_pred_cls))
print("\nClassification Report:\n", classification_report(y_test_cls, y_pred_cls))
print("\nConfusion Matrix:\n", confusion_matrix(y_test_cls, y_pred_cls))
```

```
Accuracy Score: 0.736

Classification Report:
               precision    recall  f1-score   support

   Excellent       0.81      0.34      0.48        65
        Fail       0.81      0.76      0.78       410
        Good       0.79      0.84      0.81       598
        Pass       0.59      0.63      0.61       427

    accuracy                           0.74      1500
   macro avg       0.75      0.64      0.67      1500
weighted avg       0.74      0.74      0.73      1500


Confusion Matrix:
 [[ 22   0  43   0]
 [  0 311   5  94]
 [  5   1 500  92]
 [  0  73  83 271]]
```

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Linear Kernel
svc_linear = SVC(kernel='linear', random_state=42)
svc_linear.fit(X_train_cls, y_train_cls)
y_pred_linear = svc_linear.predict(X_test_cls)
print(f"Accuracy Score with Linear Kernel: {accuracy_score(y_test_cls, y_pred_linear):.4f}")

# Sigmoid Kernel
svc_sigmoid = SVC(kernel='sigmoid', random_state=42)
svc_sigmoid.fit(X_train_cls, y_train_cls)
y_pred_sigmoid = svc_sigmoid.predict(X_test_cls)
print(f"Accuracy Score with Sigmoid Kernel: {accuracy_score(y_test_cls, y_pred_sigmoid):.4f}")

# Polynomial Kernel
```

```
svc_poly = SVC(kernel='poly', random_state=42)
svc_poly.fit(X_train_cls, y_train_cls)
y_pred_poly = svc_poly.predict(X_test_cls)
print(f"Accuracy Score with Polynomial Kernel: {accuracy_score(y_test_cls, y_pred_poly):.4f}")
```

```
Accuracy Score with Linear Kernel: 0.7400
Accuracy Score with Sigmoid Kernel: 0.7333
Accuracy Score with Polynomial Kernel: 0.6827
```